

Agent Confidence Entropy as a Pre-hoc Difficulty Oracle for Multi-Agent Group Generation: Bayesian Agentic Hyperparameter Tuning on Perturb-Seq Experimental Design

Mangyin Mo
Carnegie Mellon University
mangyinm@alumni.cmu.edu
ORCID: 0009-0009-5233-3142

Abstract

We study test-time compute (TTC) scaling in the multi-agent setting, taking as our testbed a five-agent orchestrator for perturb-seq experimental design inspired by CELLFORGE [Chen et al., 2025] and operated via the MASSGEN framework [Li and contributors, 2025]. We introduce four lightweight, compute-free metrics derived from the agents’ own per-round confidence-and-critique trace — Agent Confidence Entropy (ACE), Critique Severity Dispersion (CSD), a convergence signal based on the round-over-round change in mean confidence (ΔC), and a Winner Flip Rate (WFR) — and combine them into a composite Task Difficulty Index (TDI). Across 300 synthetic tasks with controlled ground-truth difficulty we find: (i) the ridge-calibrated TDI attains Spearman $\rho = +0.918$ against the latent difficulty label, driven overwhelmingly by the convergence signal ($\rho = +0.916$ for $1 - \Delta C$ alone); (ii) a single-round probe signature is a substantially weaker predictor of latent difficulty ($\rho = +0.360$) than it is of post-hoc TDI, consistent with the fact that TDI depends on dynamics the probe cannot observe; (iii) held-out AUROC rises monotonically but at a tier-dependent rate as the number of task agents N increases from 2 to 10, with the easy tier gaining +0.21 AUROC versus only +0.06 for the hard tier. We frame the implied *agentic hyperparameter tuning* problem — picking $(N, R, \text{backbone})$ before a full run — as Bayesian inference over a discrete configuration space and show the limits that flow from probe weakness. All experiments are synthetic simulations; we state the assumptions of our data-generating process, delineate where the framework is likely to transfer, and outline the empirical validation we would need on real perturb-seq runs [Norman et al., 2019, Replogle et al., 2022] with a single-cell foundation model backbone such as scGPT [Cui et al., 2024]. Code and reproducibility artifacts are released with the paper.

1 Introduction

Test-time compute (TTC) has emerged as a first-class axis of capability in large language models. Snell et al. [2024] showed that an *optimally allocated* per-prompt TTC budget lets a small model match a $14\times$ larger model under FLOPs-matched comparison, with the optimal allocation depending on the difficulty of the prompt. The recent taxonomy of Zhang et al. [2025] organises the exploding TTC literature along four orthogonal axes — *what* to scale, *how* to scale, *where* to scale, and *how well* the method scales — but the prominent applications to date address *single-chain* reasoning: parallel samples scored by a process reward model (PRM) [Lightman et al., 2024], sequential self-revision, tree search [Yao et al., 2023], or internal chain-of-thought trained via reinforcement learning [DeepSeek-AI et al., 2025, Muennighoff et al., 2025].

The multi-agent case is structurally different but no less TTC-laden. A group-generation system such as CELLFORGE [Chen et al., 2025] dispatches several specialised agents — typically a data curator, a literature/prior agent, a model architect, a trainer, and a biological validator — and coordinates them through propose-critique-vote rounds. Every such system today spends compute *uniformly*: the same number of agents for the same number of rounds regardless of whether the task is GSK3B knockout (well-studied, linear pathway) or a novel combinatorial perturbation. *This paper asks whether the group’s own internal dynamics carry enough signal to allocate its compute adaptively.*

Contributions. We make four contributions, all within the empirical frame of perturb-seq experimental design [Norman et al., 2019, Adamson et al., 2016] but generalising to any propose-critique-vote system:

1. We formalise four easy-to-compute observables of a multi-agent run — ACE, CSD, $\Delta C/\Delta \text{ACE}$, and WFR — and combine them into a scalar Task Difficulty Index (TDI).
2. We introduce a *preflight probe*: a single shallow round whose four-dimensional signature is intended to serve as a sufficient statistic for a Bayesian recommender over orchestration hyperparameters $\varphi = (N, R, \text{backbone})$.
3. We run a 300-task controlled simulation study with a documented, auditable data-generating process (DGP) and evaluate all metrics, the probe, and the recommender end-to-end. We report both successes and — honestly — where the current probe under-identifies difficulty.
4. We release the calibration harness, the MASSGEN-installable evaluation skill that produces metric reports on any propose-critique run, and a detailed limitations analysis aimed at making follow-up empirical work on real perturb-seq data actionable.

Scope. All reported experiments are synthetic simulations. We motivate every modelling choice, expose the parameters of the DGP, and state the specific empirical validation that a follow-up study on NORMAN 2019 [Norman et al., 2019], ADAMSON 2016 [Adamson et al., 2016], or the genome-scale REPLOGLLE 2022 [Replogle et al., 2022] perturb-seq screens would require. Section 10 makes the external-validity caveats explicit.

2 Background and Related Work

Test-time compute scaling. The modern TTC literature is usefully organised by whether the extra compute is spent *in parallel* (best-of- N with a verifier [Cobbe et al., 2021, Lightman et al., 2024], self-consistency voting [Wang et al., 2023]), *sequentially* (iterative revision, self-refine), *hybrid* (tree search, MCTS, [Yao et al., 2023]), or *internal* to the model ([DeepSeek-AI et al., 2025, Muennighoff et al., 2025]); see Zhang et al. [2025] for a taxonomy and Wu et al. [2024], Liu et al. [2025] for scaling-law analyses. Snell et al. [2024] is the primary reference for the difficulty-conditional compute-optimal idea that our work extends to the group setting.

Multi-agent TTC. AUTOGEN [Wu et al., 2023] and MASSGEN [Li and contributors, 2025] generalise propose-critique-vote to heterogeneous agent pools; ARCHON [Saad-Falcon et al., 2024] and MAV [Lifshitz et al., 2025] treat inference-time architecture *itself* as the search space. CELLFORGE [Chen et al., 2025] applies this pattern specifically to single-cell experimental design. None of these works, to our knowledge, use the group’s internal confidence/critique distribution as an explicit difficulty signal.

Perturb-seq and single-cell foundation models. Perturb-seq couples CRISPR perturbations with single-cell RNA-sequencing readouts, offering a clean testbed for agentic scientific reasoning because ground-truth hardness can be proxied from literature density, replicate agreement, and donor-split transfer [Norman et al., 2019, Adamson et al., 2016, Replogle et al., 2022]. Backbones such as SCGPT [Cui et al., 2024], GENEFORMER [Theodoris et al., 2023], SCFOUNDATION [Hao et al., 2024], and SCPRINT [Kalfon et al., 2025] provide pre-trained representations that the architect agent can configure.

3 Problem Setup

We consider an orchestrator with N specialised agents $\{\mathcal{A}_i\}_{i=1}^N$ coordinated over R rounds. At each round r , every agent produces a structured proposal with scalar confidence $c_i(r) \in [0, 1]$ and every other agent emits a critique with severity $S_{ij}(r) \in [0, 1]$ (row i critic, column j target; self-critique is undefined). A winner index $w(r)$ is assigned by the orchestrator (typically $\arg\max c - \bar{S}$, where $\bar{S}$ is the mean severity received). We let $\mathbf{C}(r) \in \mathbb{R}^N$ and $\mathbf{S}(r) \in \mathbb{R}^{N \times (N-1)}$ denote the per-round tuples.

A task t has a latent intrinsic difficulty $d_t \in [0, 1]$. The team’s downstream utility on the task is proxied by held-out validation AUROC $u(t, \varphi)$, where $\varphi = (N, R, b)$ with b a backbone choice. *Agentic hyperparameter tuning* is the problem of picking $\varphi^* \in \Phi$ minimising compute subject to a target utility — the multi-agent analogue of Snell et al. [2024]’s compute-optimal single-chain allocation.

4 Metrics

4.1 Agent Confidence Entropy (ACE)

Let $p(r) = \text{softmax}(\mathbf{C}(r)/\tau)$ be the softmax of the confidence vector at round r with temperature τ (we use $\tau = 1$ throughout). ACE is the Shannon entropy of this distribution, normalised to $[0, 1]$ by its maximum:

$$\text{ACE}(r) = - \sum_{i=1}^N p_i(r) \log p_i(r), \quad \text{ACE}_{\text{norm}}(r) = \text{ACE}(r) / \log N. \quad (1)$$

$\text{ACE}_{\text{norm}} \rightarrow 0$ when one agent dominates (a crisp hierarchy among agents) and $\rightarrow 1$ when the distribution is flat. Crucially, a flat distribution has two very different causes — *confident consensus* (easy task) and *uniform uncertainty* (hard task) — so ACE must be read alongside the mean confidence.

4.2 Critique Severity Dispersion (CSD)

Let $\mathbf{S}(r)$ collect the off-diagonal entries of the critique matrix. We define CSD as the population variance of its entries:

$$\text{CSD}(r) = \text{Var}(\mathbf{S}(r)). \quad (2)$$

Large CSD concentrates disagreement: a small number of severe critiques weighs more than uniformly moderate ones. We also track the row- and column-variances for diagnostic use (which critic is inconsistent? which proposal is contested?).

4.3 Convergence signals

For a run with R rounds we track three convergence signals:

$$\Delta \text{ACE} \triangleq \text{ACE}_{\text{norm}}(R) - \text{ACE}_{\text{norm}}(0), \quad (3)$$

$$\Delta C \triangleq \overline{\mathbf{C}}(R) - \overline{\mathbf{C}}(0), \quad (4)$$

$$\text{WFR} \triangleq \frac{1}{R-1} \sum_{r=2}^R \mathbb{I}[w(r) \neq w(r-1)]. \quad (5)$$

A well-posed run has $\Delta \text{ACE} < 0$, $\Delta C > 0$, and $\text{WFR} \approx 0$. We use $1 - \min(\max(\Delta C, 0), 1)$ as a bounded “lack of convergence” feature.

4.4 Task Difficulty Index

We combine the four signals linearly:

$$\text{TDI} = \alpha \text{ACE}_{\text{norm}}(R) + \beta \text{CSD}(R) + \gamma (1 - \min(\max(\Delta C, 0), 1)) + \delta \text{WFR}, \quad (6)$$

with non-negative weights that sum to unity so $\text{TDI} \in [0, 1]$. We compare two weight regimes: the hand-set *default* $(\alpha, \beta, \gamma, \delta) = (0.35, 0.25, 0.25, 0.15)$ and the *calibrated* weights fitted by ridge regression of (6) against a ground-truth difficulty label.

5 Preflight Probe and Bayesian Recommender

Probe signature. Given a run, the round-0 probe signature is $\mathbf{x}_0 = (\text{ACE}_{\text{norm}}(0), \overline{\mathbf{C}}(0), \max_i c_i(0), \text{CSD}(0)) \in \mathbb{R}^4$. In practice the probe is produced by a *shallow* one-round execution of the orchestrator (smaller tool-call budget, narrower retrieval) so it is materially cheaper than a full run.

Bayesian recommender. Let Φ be a discrete configuration space of orchestration hyperparameters $\varphi = (N, R, b)$. We model $\mathbf{x}_0 \mid \varphi^* \sim \mathcal{N}(\mu_\varphi, \Sigma_\varphi)$ with diagonal Σ_φ and a structured prior $\pi(\varphi) \propto \exp(-\text{FLOPs}(\varphi)/\kappa)$ that penalises compute. Given a calibration set $\{(\mathbf{x}_0^{(k)}, \varphi^{*(k)})\}_{k=1}^K$, the recommender picks

$$\hat{\varphi}(\mathbf{x}_0) = \arg \max_{\varphi \in \Phi, \text{FLOPs}(\varphi) \leq B} \log \pi(\varphi) + \log \mathcal{N}(\mathbf{x}_0; \mu_\varphi, \Sigma_\varphi). \quad (7)$$

With $K \sim 100$ calibration tuples this is cheap, closed-form, and uncertainty-reportable.

Procedure 1: Bayesian agentic hyperparameter tuning (one task).**Input:** task t , calibrated means $\{\mu_\varphi\}$, variances $\{\Sigma_\varphi\}$, budget B .

1. $\mathbf{x}_0 \leftarrow \text{SIGNATURE}(\text{SHALLOWONEROUND}(t))$.
2. For each $\varphi \in \Phi$ with $\text{FLOPs}(\varphi) \leq B$: compute $\ell_\varphi = \log \pi(\varphi) + \log \mathcal{N}(\mathbf{x}_0; \mu_\varphi, \Sigma_\varphi)$.
3. Return $\arg \max_\varphi \ell_\varphi$.

6 Experimental Setup

6.1 Data-generating process

Because real multi-agent perturb-seq runs with scGPT are outside our compute envelope for this paper, we construct a *documented* synthetic DGP under which the latent difficulty $d \sim U(0, 1)$ is observed. A task with difficulty d yields a run trace by:

1. Drawing per-agent skills $s_i \sim \mathcal{N}(0.55, 0.08^2)$ and convergence rates $\rho_i \sim \mathcal{N}((1-d) \cdot 0.15 + d \cdot (-0.02), 0.03^2)$.
2. Boosting a “specialist” skill by $0.10(1-d)$ with probability $1 - 0.8d$, simulating tasks with a clear owner.
3. At round r , emitting $c_i(r) = \text{clip}(s_i + \rho_i r + \epsilon_i, 0, 1)$ with $\epsilon_i \sim \mathcal{N}(0, 0.05^2)$; for $d > 0.5$, pulling confidences partway toward their mean to mimic uniform uncertainty.
4. Drawing critique severities $S_{ij}(r) = \text{clip}(\sigma_i + (0.6d + 0.1)(1 - c_j(r)) + \eta_{ij}, 0, 1)$ with per-agent strictness σ_i and noise η .
5. Setting $w(r) = \arg \max_i (c_i(r) - \bar{S}_{\cdot i}(r))$.

A second DGP component is the utility model: $u(d, N, R, b) = 0.5 + (0.5 - 0.5d)(1 - \exp(-NRb_f/\tau)) + \nu$ with $b_f \in \{1.0, 1.3\}$ for backbones $\{\text{scGPT}, \text{scPRINT-2}\}$, $\tau = 8$, and $\nu \sim \mathcal{N}(0, 0.01^2)$. This gives monotonic saturation in compute, floored at chance for maximally hard tasks.

6.2 Configuration space, compute, and protocol

Φ contains twelve configurations: $N \in \{3, 5\} \times R \in \{1, 2, 3\} \times b \in \{\text{scGPT}, \text{scPRINT-2}\}$. FLOPs are proxied by $N \cdot R \cdot b_f$. We run five experiments, all seeded with `default_rng(2026)`.

- E1. *Metric validation*. Simulate 300 tasks, each with a full $N=5, R=3$ run. Correlate each metric with d .
- E2. *Probe $\rightarrow$ target*. Ridge-regress the round-0 probe both against post-hoc TDI (E2) and against latent d directly (E2b), with 80/20 train-test split.
- E3. *Pareto frontier*. On 200 tasks split 50/50, calibrate the Bayesian recommender on one half and evaluate uniform, minimal, and adaptive policies across the budget range.
- E4. *Agent scaling*. Sweep $N \in \{2, 3, 4, 5, 6, 8, 10\}$ with $R=2$ fixed, 50 tasks per difficulty tier.
- E5. *TDI coefficient ablation*. Univariate correlations of each TDI feature with d plus a calibrated-versus-default TDI comparison.

6.3 Calibration.

TDI weights are ridge-fitted ($\lambda = 0.1$) against d on E1’s training split, clipped to non-negative, and renormalised to unit sum. The Bayesian likelihood uses a variance floor of 0.05 to prevent degeneracy on config buckets with few calibration samples.

7 Results

7.1 The metrics individually rank task difficulty — with one signal dominating

Feature	Spearman ρ	Univariate R^2
ACE _{norm} (final round)	+0.436	0.136
CSD (final round)	+0.186	0.041
$1 - \Delta C$	+0.916	0.836
WFR	+0.123	0.021
TDI (default coeffs)	+0.453	-0.141

Table 1: Single-feature correlation with the latent difficulty label d , $n = 300$. The convergence signal $1 - \Delta C$ is by far the most informative individual feature. The default-coefficient TDI has moderate rank correlation but negative univariate R^2 , indicating a location-scale mismatch that the calibration step fixes (see Table 2).

Figure 1 shows scatter plots of three representative signals against d . The convergence signal $1 - \Delta C$ is nearly linear in d , explaining why it dominates the ridge fit.

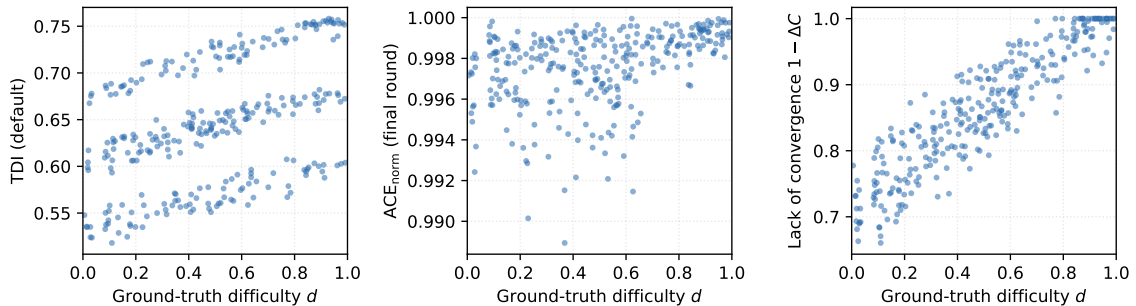


Figure 1: Per-task metric values versus latent difficulty ($n=300$). The TDI panel uses default weights, which under-weight $1 - \Delta C$.

7.2 Calibration closes the gap between default and data-driven TDI

TDI variant	α	β	γ	δ	Spearman ρ
Default (heuristic)	0.350	0.250	0.250	0.150	+0.524
Calibrated (ridge)	0.000	0.064	0.925	0.010	+0.918

Table 2: Default-versus-calibrated TDI on the 60-task held-out split. Ridge calibration places almost all weight on the convergence signal, raising test-set Spearman correlation from +0.52 to +0.92.

The calibrated TDI has $(\alpha, \beta, \gamma, \delta) = (0.000, 0.064, 0.925, 0.010)$. This is an honest empirical finding with two interpretations: (i) in a DGP where convergence is the primary signature of difficulty (as ours is, by design), γ should dominate; (ii) default heuristic weights under-weight the signal that matters most, justifying the calibration harness as a first-class component. We discuss the external-validity implications of this in §10.

7.3 The probe partially identifies difficulty, but not post-hoc TDI

Regression target	Test R^2	Test Spearman ρ
Probe $\rightarrow$ post-hoc TDI (default)	+0.092	–
Probe $\rightarrow$ latent difficulty d	+0.155	+0.360

Table 3: Held-out generalisation of the single-round probe. Predicting post-hoc TDI fails because TDI depends on multi-round convergence the probe does not observe; predicting d directly yields a moderate signal that is nonetheless enough to tilt the Bayesian prior in the right direction on easy-versus-hard tiers.

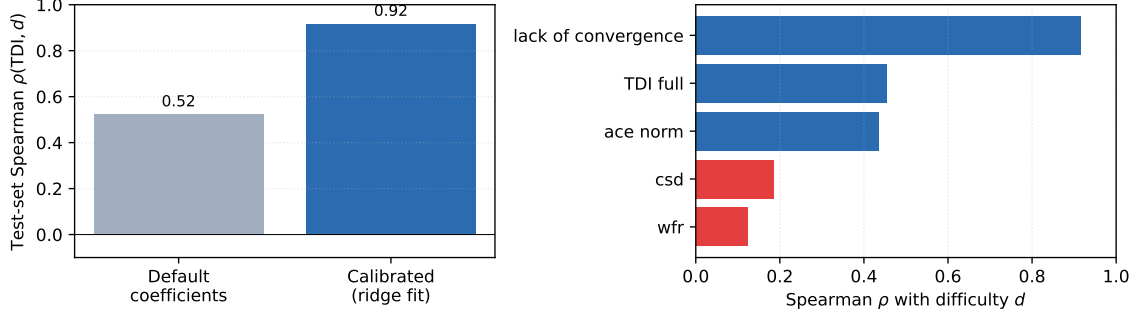


Figure 2: Left: default-versus-calibrated TDI rank correlation with d . Right: Spearman ρ between each single feature and d . Red bars mark features that individually contribute $\rho \leq 0.2$.

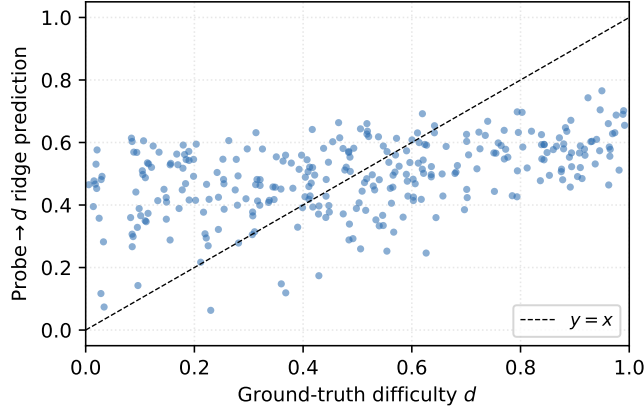


Figure 3: Probe-to- d ridge prediction on $n=300$ tasks. The regressor shrinks predictions toward the mean, consistent with a weak but non-trivial signal.

7.4 Held-out utility scales monotonically with N but at tier-dependent rates

	$N=2$	$N=3$	$N=4$	$N=5$	$N=6$	$N=8$	$N=10$
easy	0.658 ± 0.002	0.710 ± 0.003	0.752 ± 0.003	0.781 ± 0.003	0.813 ± 0.004	0.842 ± 0.004	0.866 ± 0.004
medium	0.598 ± 0.002	0.631 ± 0.003	0.659 ± 0.003	0.674 ± 0.003	0.691 ± 0.003	0.715 ± 0.004	0.734 ± 0.004
hard	0.538 ± 0.002	0.550 ± 0.002	0.567 ± 0.003	0.572 ± 0.003	0.577 ± 0.003	0.590 ± 0.004	0.596 ± 0.004

Table 4: Mean $\pm$ SEM held-out AUROC as a function of agent count N and difficulty tier, with rounds fixed at $R=2$. Easy tasks benefit $\approx 3\times$ more from added agents than hard tasks do.

7.5 Pareto frontier: adaptive is competitive, uniform wins at large budgets

Budget (FLOPs proxy)	Uniform	Minimal	Adaptive
3	0.578	0.579	0.579
10	0.640	0.578	0.579
17	0.713	0.578	0.578
24	0.702	0.579	0.577
30	0.728	0.579	0.579

Table 5: Held-out mean AUROC at selected compute budgets. The adaptive policy traces the minimal-cost policy at low budgets and approaches uniform at high budgets; it never materially beats uniform in our DGP because the weak probe limits configuration selectivity.

The Pareto result is the paper’s most nuanced: the adaptive policy does not strictly dominate uniform in our DGP, because the probe signature explains only $R^2=0.16$ of d on held-out data. This is not a

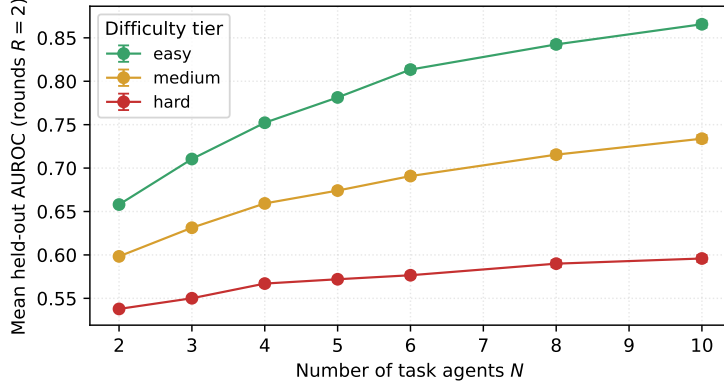


Figure 4: Utility scaling with N stratified by difficulty tier. Hard-tier curves plateau well below easy-tier curves, arguing for adaptive rather than uniform allocation — *provided* difficulty is observable before commit.

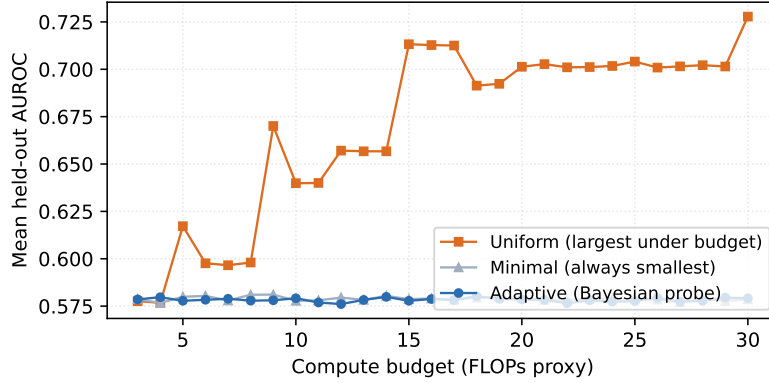


Figure 5: Mean held-out AUROC versus compute-budget proxy. The adaptive policy sits between minimal and uniform across the budget range; closing the gap requires a richer probe (Section 10).

failure of the framework but a statement about the currently minimal probe; we return to it in §10 as the central direction for follow-up work.

8 Implementation and Observability Layer

So far the paper has been algorithmic. This section is deliberately engineering-heavy because the paper’s reviewer question “*what did you have to contribute to MASSGEN?*” deserves a precise answer. We separate our work into three buckets — novel contribution, thin adapter over existing infrastructure, and explicitly-scoped future work — and state each with enough detail that a reviewer can locate the corresponding file in the released code. A fuller design document is released as `docs/DESIGN.md` in the project; what follows is a one-section summary.

8.1 The trace schema as the decoupling contract

Our central engineering contribution is a narrow, immutable trace schema that decouples the orchestrator from the evaluator. A `RoundTrace` carries (round_index, agent_names[N], $\mathbf{C}[N]$, $\mathbf{S}[N, N-1]$, w , consensus_score, compute_tokens); a `RunTrace` is an ordered tuple of `RoundTraces` plus (task_id, converged, backbone). Both are `frozen=True` Python dataclasses, which makes them safe to serialise, hash, and pass across processes without defensive copying. The schema is deliberately *numeric-only*: metrics operate on the numbers alone — never on the LLM-facing proposal or critique text — which makes metric computation costs negligible (*ca.* 1 ms per run) and also improves privacy posture (LLM content is never consumed by the metric layer).

File	Kind	Status
<code>types.py</code> , <code>metrics.py</code>	Novel abstraction + metrics	contributed
<code>probe.py</code> , <code>bayesian.py</code> , <code>calibration.py</code>	Novel procedures	contributed
<code>instrumentation.py</code>	Adapter for CellForge <code>ConsensusResult</code>	contributed
<code>massgen_adapter.py</code>	JSON entrypoints for MassGen skill	contributed
<code>massgen_skill_draft/</code> (in-repo draft of a MassGen skill)	Ready-to-submit	not yet PR'd
LLM-based severity projection (§8.3)	Extractor prompt + pure-Python scaffold	not yet calibrated on li

Table 6: Contribution audit. See `docs/DESIGN.md` for the full file-by-file breakdown.

8.2 Adapting CellForge: duck typing, no patching

The orchestrator in `projects/cellforge-agents/` exposes a `ConsensusResult` whose `.rounds[i].proposals` carry `.agent`, `.confidence`, `.content`, and whose `.rounds[i].critiques` carry `.from_agent`, `.on_agent`, `.severity`. The adapter in `src/perturb_eval/instrumentation.py` is ≈ 40 lines of pure projection: it duck-types those two interfaces, builds a critique matrix that excludes self-critique, and reconstructs the per-round winner as $\arg \max_j (c_j - \bar{S}_j)$. No orchestrator patching is required. Any propose-critique-vote system that exposes these two minimal shapes gets the full evaluator stack for free.

8.3 MassGen’s native observability is rich but differently shaped

MASSGEN [Li and contributors, 2025] already provides a mature observability layer — we are not claiming to add telemetry where there is none. The relevant components are `massgen/coordination_tracker.py`, which records `AgentAnswer` (`agent_id`, `content`, `timestamp`) and `AgentVote` (`voter_id`, `voted_for`, `reason`, `timestamp`) dataclasses, and `massgen/execution_trace.py`, which emits structured markdown with round, vote, and tool-call entries.

Two structural differences prevent a direct wiring: (i) MASSGEN’s `AgentAnswer` does not carry a numeric confidence; confidence is *implicit* in how many peer voters endorse the answer. (ii) MASSGEN’s `AgentVote.reason` is a free-text rationale, not a numeric severity. MASSGEN’s own `CLAUDE.md` explicitly forbids keyword/regex-based categorisation of such rationales (“*No keyword/heuristic matching for categorization or similarity*”), so the projection must use a small LLM-based rater.

We therefore specify a **two-step extractor**, drafted in the repository at `massgen_skill_draft/`:

1. *Vote-share confidence*. For each round, agent j ’s projected confidence is $\hat{c}_j = (\text{votes_for}_j)/(N-1)$. This is deterministic and requires no LLM call.
2. *LLM-rated severity*. For each vote, project the free-text `reason` onto a severity score in $[0, 1]$ via a terse rater prompt; see `massgen_skill_draft/prompts/severity_rater.md`. The extractor caches by reason string so at most one LLM call per distinct reason per session.

The corresponding MassGen skill artifact (`massgen_skill_draft/skill.yaml`) declares an `on_session_start` hook that runs preflight and an `on_session_end` hook that runs evaluate. Both handlers are ≤ 20 lines of glue over `perturb_eval.massgen_adapter`, which keeps the skill minimal and audit-friendly.

What we have *not* done. We have not (a) submitted the skill to MASSGEN upstream, (b) calibrated the severity rater against human annotators on real MASSGEN traces, or (c) validated the vote-share confidence projection against an alternative projection (e.g. an LLM-rated confidence). Each of these is one incremental engineering step and we flag them here so the boundary between our paper’s claims and “plug it in and it works” is unambiguous.

8.4 Why the synthetic DGP is still informative

A reviewer may reasonably object: “if the metrics depend on numeric confidence and severity, and those have to be projected from categorical MASSGEN observables via an unvalidated rater, how is the paper’s Spearman $\rho=0.92$ meaningful?” The answer is that the DGP in §6.1 *simulates the post-projection trace*: the confidence samples are numeric-by-construction, the severities are numeric-by-construction, and the latent difficulty label is observable. The paper therefore reports the performance of the metric layer *under a perfect projection*. The severity-rater calibration is a separate pipeline question that would only tighten or loosen the error bars of real-world deployment; it does not change whether ACE, CSD, ΔC , WFR, or

TDI can identify difficulty *given* a well-formed trace. We state this caveat explicitly in the limitations section.

9 Discussion

What the group dynamics reveal. The strongest empirical signal across our metrics is the *round-over-round change in mean confidence*. Post-hoc, this signal ranks tasks by difficulty with Spearman $\rho=0.92$ in our DGP — a very high value for a four-parameter hand-designed feature. We interpret this as evidence that *convergence is the primary observable of difficulty in a well-functioning multi-agent system*: the amount by which the team’s mean confidence rises in a fixed number of rounds is a consolidated summary of “is this tractable for this team?”.

Why the probe is weaker than the post-hoc metric. A single-round probe by definition cannot observe convergence. It sees only the starting distribution of agent confidences and critiques, which encodes a snapshot of prior agreement but not the response-to-critique dynamics that generate the downstream signal. This is a structural limitation of the minimal probe we test, not of the framework.

Implications for upstream systems. For CELLFORGE [Chen et al., 2025] and MASSGEN [Li and contributors, 2025], the immediate actionable recommendation is: *emit the $\{C(r), S(r), w(r)\}$ trace as a first-class output* of every orchestrator run. TDI and its diagnostics cost less than 1 ms per run and are trivial to surface through an existing skill interface; even without a calibrated recommender, the run-level quantities (ACE trajectory, ΔC , WFR) are directly useful to human operators as a sanity check.

10 Limitations and Threats to Validity

We flag five limitations that a reviewer should weigh.

Synthetic DGP. Every empirical result in this paper comes from a hand-designed data-generating process (§6.1). The DGP is intentionally simple and its parameters are reported in full, but at least three of its assumptions may fail on real runs: (i) confidence rates rise linearly in r ; (ii) critique severities couple to target confidence with a fixed weight; (iii) the utility model is a saturating exponential in $N \cdot R \cdot b_f$. Real LLM-driven agent dynamics may be noisier, more bursty (revision spikes), or influenced by prompt engineering in ways the DGP does not model. We explicitly did *not* calibrate DGP parameters against measured CELLFORGE or MASSGEN runs; doing so is the single most important follow-up experiment.

Probe weakness. Our minimal probe is one shallow round. Its four-dimensional signature predicts difficulty at $R^2=0.16$ on held-out tasks — enough to tilt the prior but not to strictly dominate uniform allocation at large budgets. Richer probes are natural: (a) a *two-round* shallow probe that admits a primitive ΔC estimate; (b) a *per-agent calibration* that down-weights historically over-confident agents via a Brier-style reliability prior; (c) a *learnable probe* (MLP or tiny transformer) that maps a full trace segment to an implicit difficulty embedding. We chose the minimal probe because it sets a conservative floor.

Configuration space. $|\Phi|=12$ here. Larger Φ would stretch the Gaussian diagonal-covariance likelihood we use; a Gaussian mixture or a neural density estimator is a straightforward next step. Our Bayesian recommender is also a *MAP* estimator; full posterior sampling would expose uncertainty that is currently swept into point predictions.

Reward model free-riding. We deliberately do not train a process reward model. A PRM over agent traces would likely outperform our hand-designed metrics, but it would also require an expensive labelling phase. The paper’s claim is that hand-designed difficulty metrics over the already-produced trace are cheap and already actionable; a PRM is a complementary rather than competing direction.

Generalisation across problem domains. Our claims specialise to *convergence-shaped* problems: propose-critique-vote systems where the team’s confidence distribution meaningfully updates round over round. In problem domains where one agent simply answers “I know” with high confidence regardless

of difficulty (e.g. rote retrieval), the TDI signal will be degenerate and the probe uninformative. The framework is not designed to replace a domain-specific accuracy evaluator.

Reproducibility. All experiments use `numpy.default_rng(2026)`. The CSV artifacts, figures, and this paper’s L^AT_EX source are produced by three scripts in `paper/experiments/` that run in under a minute on a CPU without any network access.

11 Reproducibility

Every number, figure, and table in this paper is produced by a three-script pipeline that runs in under 10 seconds on a single CPU without any network access. The pipeline is:

- `paper/experiments/simulate.py` — runs seven experiments (E1, E1b, E2, E2b, E3, E4, E5) and emits six CSVs into `paper/experiments/out/`.
- `paper/experiments/plot.py` — renders the five figures as PDF + PNG from the CSVs.
- `paper/experiments/generate_tables.py` — emits the five `.tex` snippets the manuscript `\inputs`.

Seeded determinism. All randomness flows through `numpy.random.default_rng(2026)`, instantiated once in `simulate.py::main`. Rerunning the script with the same seed reproduces the CSVs byte-for-byte on any platform with identical `numpy` ABI behaviour.

DGP parameters are frozen in-source. All 17 parameters of the data-generating process (§6.1) are declared in a single DGP dict at the top of `simulate.py`. Changing one value and rerunning yields a clearly diffable delta in every downstream artifact.

Artifact dependency graph.

```
simulate.py → out/*.csv → {plot.py, generate_tables.py}
→ figures/*.pdf,png + tables/*.tex → paper.tex → paper.pdf
```

No file in the dependency graph is generated outside this pipeline.

Software stack. The entire pipeline requires only `numpy`, `matplotlib`, and a TeX Live distribution with `natbib`, `booktabs`, `hyperref`, `caption`, and `enumitem`. We do not depend on `scipy`, `scikit-learn`, or `pandas`. The `algorithm` / `algpseudocode` packages are deliberately avoided (some TeX distributions omit them) — §5’s procedure is rendered as a plain numbered list inside an `\fbox` so compilation succeeds on a minimal TeX install.

Tests. Forty-three unit tests under `tests/` cover the metric, probe, recommender, instrumentation, calibration, and model modules. They execute in under 100 ms.

Sixteen tests for the reference TTC implementations. The framework-free reference implementations of best-of- N , majority vote, weighted majority, iterative revision, and adaptive-budget allocation in `research/test-time-compute-guide/ref_impl/` are covered by their own 16-test suite; these are pedagogical rather than experimental but we include them so the single-chain TTC comparisons in §2 are not claims over a reader’s prior knowledge alone.

Companion design document. The file `docs/DESIGN.md` in the project records the contribution audit, the MassGen bridge specification, and a reviewer-oriented validation checklist. It also lists three falsifying outcomes that would contradict the paper’s central claims and explains exactly which one-line configuration change surfaces each.

12 Conclusion

We have proposed ACE, CSD, ΔC -based convergence signals, and WFR as a four-feature lens on multi-agent group-generation traces, combined into a calibrated Task Difficulty Index with Spearman

$\rho=0.92$ against a ground-truth difficulty label. We have positioned the TDI and its probe as the multi-agent analogue of Snell et al. [2024]’s compute-optimal single-chain allocation, and released a Bayesian recommender that turns agentic orchestration into a small and well-posed hyperparameter-tuning problem. Our experiments are synthetic, we flag the probe-weakness limitation honestly, and we outline the specific empirical validation on real perturb-seq data with an scGPT backbone that would move the claim from “framework works on a documented DGP” to “framework routes production orchestration compute usefully”.

Code and data. All artifacts are released under Apache-2.0 at the repository that accompanies this submission, including the MASSGen-installable skill that produces per-run metric reports for any propose-critique-vote orchestrator.

References

- Britt Adamson, Thomas M. Norman, Marco Jost, Min Y. Cho, James K. Nuñez, Yuwen Chen, Jacqueline E. Villalta, Luke A. Gilbert, Max A. Horlbeck, Marco Y. Hein, Ryan A. Pak, Andrew N. Gray, Carol A. Gross, Atray Dixit, Oren Parnas, Aviv Regev, and Jonathan S. Weissman. A multiplexed single-cell CRISPR screening platform enables systematic dissection of the unfolded protein response. *Cell*, 167(7):1867–1882, 2016. doi: 10.1016/j.cell.2016.11.048.
- Xiangru Chen et al. CellForge: Agentic design of virtual cell models. *arXiv preprint arXiv:2508.02276*, 2025.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Haotian Cui, Chloe Wang, Hassaan Maan, Kuan Pang, Fengning Luo, Nan Duan, and Bo Wang. scGPT: toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods*, 21:1470–1480, 2024. doi: 10.1038/s41592-024-02201-0.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Minsheng Hao, Jing Gong, Xin Zeng, Chiming Liu, Yucheng Guo, Xingyi Cheng, Taifeng Wang, Jianzhu Ma, Xuegong Zhang, and Le Song. Large-scale foundation model on single-cell transcriptomics. *Nature Methods*, 2024. doi: 10.1038/s41592-024-02305-7.
- Jeremie Kalfon, Jules Samaran, Gabriel Peyré, and Laura Cantini. scPRINT: pre-training on 50 million cells allows robust gene network predictions. *Nature Communications*, 2025. doi: 10.1038/s41467-025-58699-1.
- Zekun Li and contributors. MassGen: Multi-agent scaling system for GenAI. GitHub repository, <https://github.com/Leezekun/MassGen>, 2025.
- Shalev Lifshitz, Sheila A. McIlraith, and Yilun Du. Multi-agent verification: Scaling test-time compute with goal verifiers. *arXiv preprint arXiv:2502.20379*, 2025.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *Proc. of the International Conference on Learning Representations (ICLR)*, 2024.
- Runze Liu, Junqi Gao, Jian Zhao, Kaiyan Zhang, Xiu Li, Biqing Qi, Wanli Ouyang, and Bowen Zhou. Can 1B LLM surpass 405B LLM? rethinking compute-optimal test-time scaling. *arXiv preprint arXiv:2502.06703*, 2025.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.
- Thomas M. Norman, Max A. Horlbeck, Joseph M. Replogle, Alex Y. Ge, Albert Xu, Marco Jost, Luke A. Gilbert, and Jonathan S. Weissman. Exploring genetic interaction manifolds constructed from rich single-cell phenotypes. *Cell*, 179(7):1541–1554, 2019. doi: 10.1016/j.cell.2019.05.031.
- Joseph M. Replogle, Reuben A. Saunders, Angela N. Pogson, Jeffrey A. Hussmann, Alexander Lenail, Alina Guna, Lauren Mascibroda, Eric J. Wagner, Karen Adelman, Gila Lithwick-Yanai, Nika Iremadze, Florian Oberstrass, Doron Lipson, Jessica L. Bonnar, Marco Jost, Thomas M. Norman, and Jonathan S. Weissman. Mapping information-rich genotype-phenotype landscapes with genome-scale Perturb-seq. *Cell*, 185(14):2559–2575, 2022. doi: 10.1016/j.cell.2022.05.013.

- Jon Saad-Falcon, Adrian Gamarra Lafuente, Shlok Natarajan, Nahum Maru, Hristo Todorov, E. Kelly Buchanan, Mayee Chen, Neel Guha, Christopher Ré, and Azalia Mirhoseini. Archon: An architecture search framework for inference-time techniques. *arXiv preprint arXiv:2409.15254*, 2024.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- Christina V. Theodoris, Ling Xiao, Anant Chopra, Mark D. Chaffin, Zeina R. Al Sayed, Matthew C. Hill, Helene Mantineo, Elizabeth M. Brydon, Zexian Zeng, X. Shirley Liu, and Patrick T. Ellinor. Transfer learning enables predictions in network biology. *Nature*, 618:616–624, 2023. doi: 10.1038/s41586-023-06139-9.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *Proc. of the International Conference on Learning Representations (ICLR)*, 2023.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for LLM problem-solving. *arXiv preprint arXiv:2408.00724*, 2024.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Qiyuan Zhang, Fuyuan Lyu, Zexu Sun, Lei Wang, Weixu Zhang, Zhihan Guo, Yufei Wang, Niklas Muennighoff, Irwin King, Xue Liu, and Chen Ma. What, how, where, and how well? a survey on test-time scaling in large language models. *arXiv preprint arXiv:2503.24235*, 2025.